

SmartFlow: Adaptive Traffic Signal Control with Reinforcement Learning

Repository: https://github.com/yq688m3JT/final_report.git

Zhonghuan Tang

Introduction

Urban traffic control is a useful setting for studying sequential decision-making because signal timing affects both the vehicles currently waiting and the congestion that will appear later. A fixed-cycle traffic signal follows the same schedule even when demand changes, which means it may waste green time on an empty lane or fail to respond when one direction becomes crowded. Adaptive control is meant to solve this problem by changing signal behavior in response to observed conditions. Reinforcement learning is a natural approach because it lets a controller improve through interaction with a traffic environment rather than relying only on a rule written in advance.

The SmartFlow project began with a simple single-intersection experiment. The goal of that first experiment was to test whether reinforcement learning could learn a useful traffic signal policy and how its performance compared with common baselines. The project then extended the same idea to a more complex two-intersection corridor. This extension is important because real traffic networks are rarely isolated. A decision at one signal can affect queues at another signal several steps later. That delayed interaction is exactly where a long-term learning method should become more valuable than a purely reactive heuristic.

Part I: Original Single-Intersection Experiment

The original experiment defined the intersection as a Markov Decision Process. The state was $S = (Q_{NS}, Q_{EW}, \text{Phase})$, where Q_{NS} is the north-south queue, Q_{EW} is the east-west queue, and Phase indicates the current green-light direction. To make tabular learning manageable, each queue was placed into one of three buckets: low traffic for 0 to 3 vehicles, medium traffic for 4 to 8 vehicles, and high traffic for 9 or more vehicles. This representation reduces the state space while still giving the agent basic information about which direction is more congested.

The action space contained two choices: Stay or Switch. Stay means the signal keeps the current green phase, while Switch means the signal changes to the other direction. The reward function was $R = -(Q_{NS} + Q_{EW}) - C$, where the first term penalizes waiting vehicles and the switching cost C discourages the controller from changing phases too frequently. This design captures the main tradeoff in signal control. The agent should reduce queues, but it should not create unrealistic or unsafe oscillation by switching at every time step.

The algorithms compared in the original presentation were Q-Learning, SARSA, Monte Carlo learning, static fixed-cycle timing, a greedy longest-queue heuristic, and Value Iteration as an optimal reference. These methods create a useful range of comparisons. Static timing represents a simple non-adaptive baseline. Greedy timing represents a strong local heuristic because it directly serves the longer current queue. Value Iteration represents the best policy available under the simplified model assumptions. The reinforcement learning methods show whether an agent can learn adaptive behavior from repeated simulated experience.

The preserved presentation results show that SARSA slightly outperformed static timing, while the greedy heuristic and Value Iteration performed better than the learned tabular agents. This result should be

interpreted carefully. SARSA beating static timing supports the basic claim that reinforcement learning can learn an adaptive signal policy. However, greedy performing strongly does not mean reinforcement learning is useless. In a single two-phase intersection, the current local queue is almost the whole problem. A longest-queue rule has direct access to that information, so it can be very effective without learning any long-term strategy.

Why the Simple Experiment Is Limited

The main limitation of the first experiment is that the environment is too local. There are no neighboring intersections, no vehicles traveling between signals, and no downstream spillback. The greedy heuristic only needs to ask which queue is longer right now. Because there is no network effect, that short-term decision often lines up with good performance. The reinforcement learning agents also used coarse buckets instead of exact queue counts, so they had less detailed state information than the greedy rule. This creates a perception gap: the heuristic appears stronger partly because it sees exact local queues in a very simple environment.

This limitation motivated the second experiment. A stronger test of reinforcement learning should include delayed consequences and coordination. In real corridors, releasing vehicles from one intersection can create a platoon that arrives at another intersection later. If the downstream light is not prepared, the released vehicles may create congestion or spillback. A local greedy controller cannot easily anticipate that future arrival if it only reacts to the queue currently visible at each intersection. A reinforcement learning controller can use a fuller state and learn that an upstream action may require a downstream response several steps later.

Part II: Coordinated Corridor Extension

The extension experiment uses a two-intersection corridor. Intersection A is upstream of Intersection B, and vehicles released from A reach B after a four-step delay. The state includes bucketed queues for both directions at both intersections, the current signal phase at A, the current signal phase at B, an in-transit vehicle bucket, and a demand-regime indicator. The action space contains four joint actions: both signals stay, A stays while B switches, A switches while B stays, or both signals switch. This creates a larger and more coordinated decision problem than the original single-intersection case.

The reward is also more system-oriented. It penalizes total queue length, phase switching, and spillback while giving credit for throughput. This is important because raw throughput alone can be misleading. A controller might release many vehicles from A, but if B is not ready to serve them, the network delay can still become worse. The experiment therefore rewards policies that manage the corridor as a whole rather than policies that only clear one visible queue at a time.

For the reinforcement learning method, the project uses a Deep Q-Network. A DQN is appropriate because the coordinated corridor has a larger state representation than the original tabular experiment. The model receives the flattened state vector and outputs Q-values for the four joint actions. Training uses experience replay, a target network, and epsilon-greedy exploration. The baselines include static fixed timing, local greedy timing, and a threshold heuristic. Static timing provides a simple control reference. Greedy timing reacts to the currently larger local queue. The threshold heuristic switches only when the opposing queue is larger by a specified margin.

Results and Interpretation

The complex corridor results support the broader argument of the project. In the coordinated setting, the DQN policy achieved lower average queue and lower average delay than static timing, local greedy timing, and the threshold heuristic. The local greedy baseline still moved many vehicles, but it also switched frequently and produced higher system delay. This pattern is meaningful because it shows the weakness of purely reactive control. Greedy can clear what it sees immediately, but it does not plan for vehicles that are already moving through the corridor and will arrive downstream in the near future.

The difference between Part I and Part II is the main lesson. In the simple intersection, greedy remains competitive because the environment is small and the current queue is highly informative. In the coordinated corridor, the current queue is no longer enough. A good policy must understand timing, delayed arrivals, and the relationship between upstream and downstream phases. Reinforcement learning becomes more valuable because it can optimize long-term system behavior instead of only reacting to the largest local queue.

This does not mean that reinforcement learning should be described as universally better than heuristic traffic control. Heuristics are often fast, interpretable, and effective in simple settings. The correct conclusion is more specific: reinforcement learning has a structural advantage when the environment contains delayed effects, coordination requirements, and network interactions that are difficult to encode in a simple rule. The extension experiment was designed around exactly those features, so it gives a more convincing reason to continue developing learning-based adaptive traffic control.

Conclusion

SmartFlow shows a clear progression from a basic MDP demonstration to a richer traffic-control experiment. The original presentation established the core formulation and showed that SARSA could improve on static timing in a single-intersection environment. The extension then showed why the value of reinforcement learning becomes clearer in a coordinated corridor, where decisions have delayed downstream consequences. Future work should expand the environment to larger road networks, continuous state representations, multi-agent signal control, and more realistic safety constraints. The project's overall conclusion is that reinforcement learning is most promising when traffic control requires coordination over time, not merely reaction to the queue visible at the present moment.

Appendix

Table A1. Preserved Presentation Results for Part I

Approach	Avg. Reward	Time
Optimal (VI)	-378.83	0.24s
Greedy	-391.93	0.22s
SARSA	-444.43	0.85s
Static	-448.48	0.20s
Q-Learning	-455.82	1.16s
Monte Carlo	-458.65	5.81s

Table A2. Complex Corridor Evaluation Results

Approach	Reward	Avg. Queue	Avg. Delay	Throughput	Switches
DQN RL	-6352.98	41.75	41.75	817.23	66.55
Static	-13452.73	78.96	81.50	854.11	28.00
Greedy	-17328.68	99.41	102.29	927.54	181.18
Threshold	-17546.14	101.53	104.56	904.86	70.49

Figure A1. Part I Reward Comparison

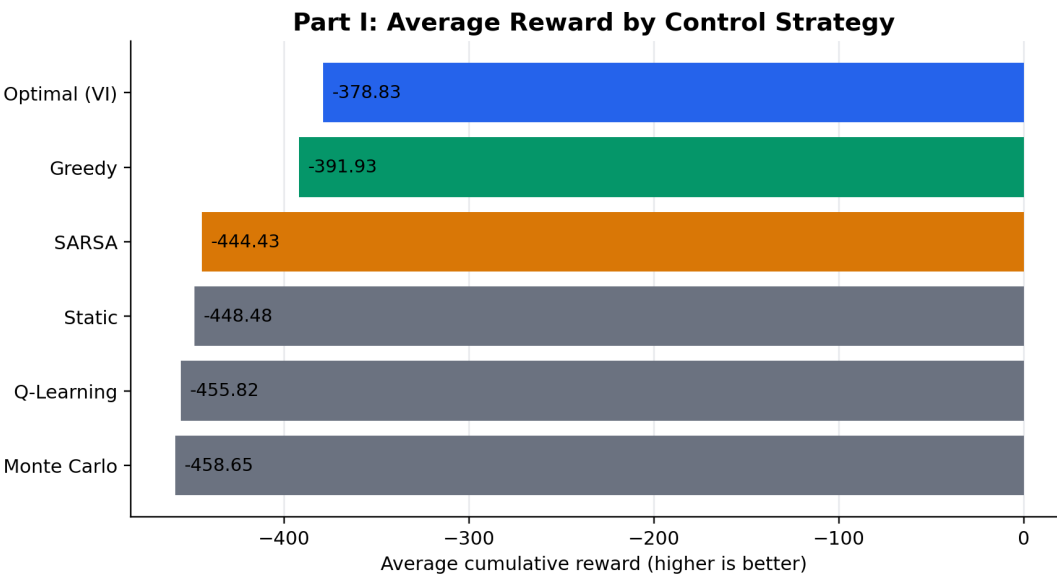


Figure A2. Part I Execution-Time Comparison

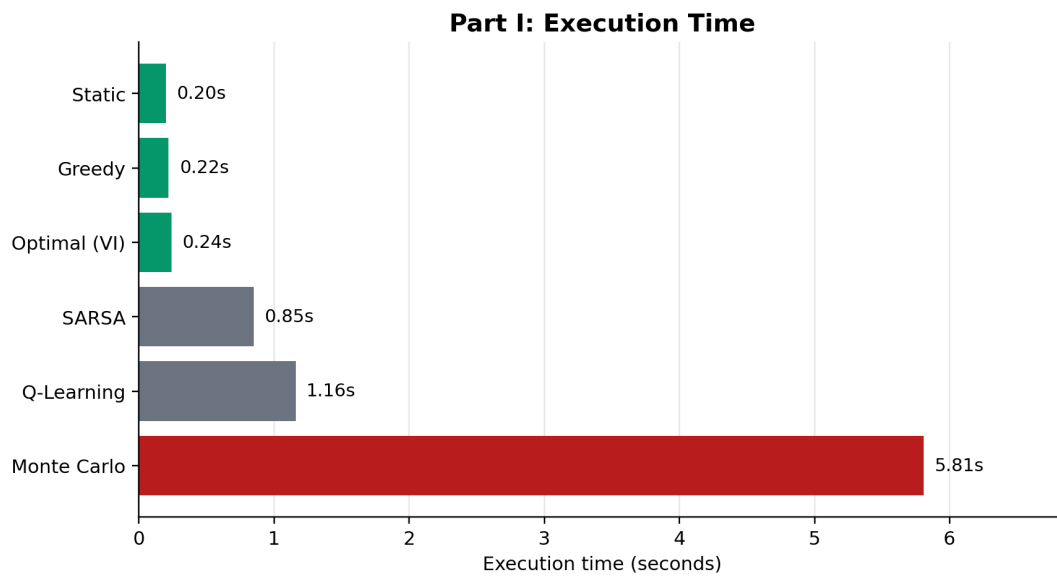


Figure A3. Coordinated Corridor Diagram

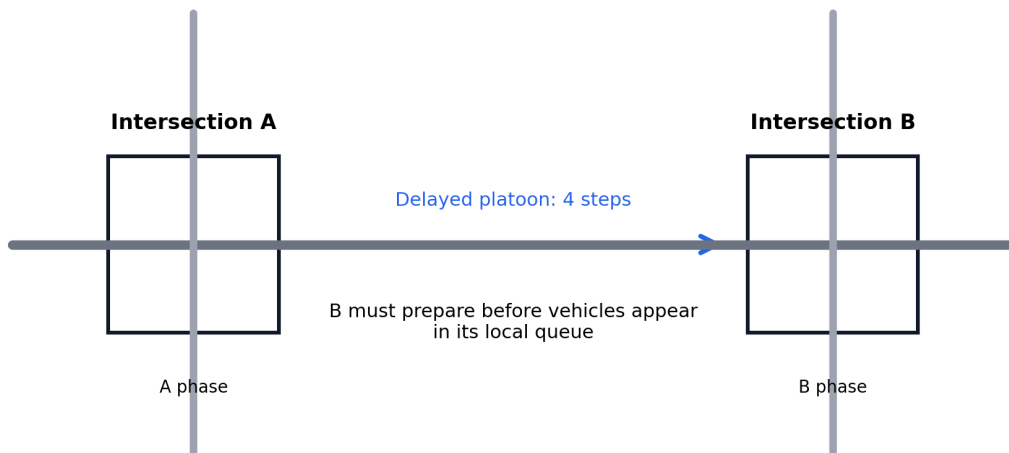


Figure A4. Part II Reward Comparison

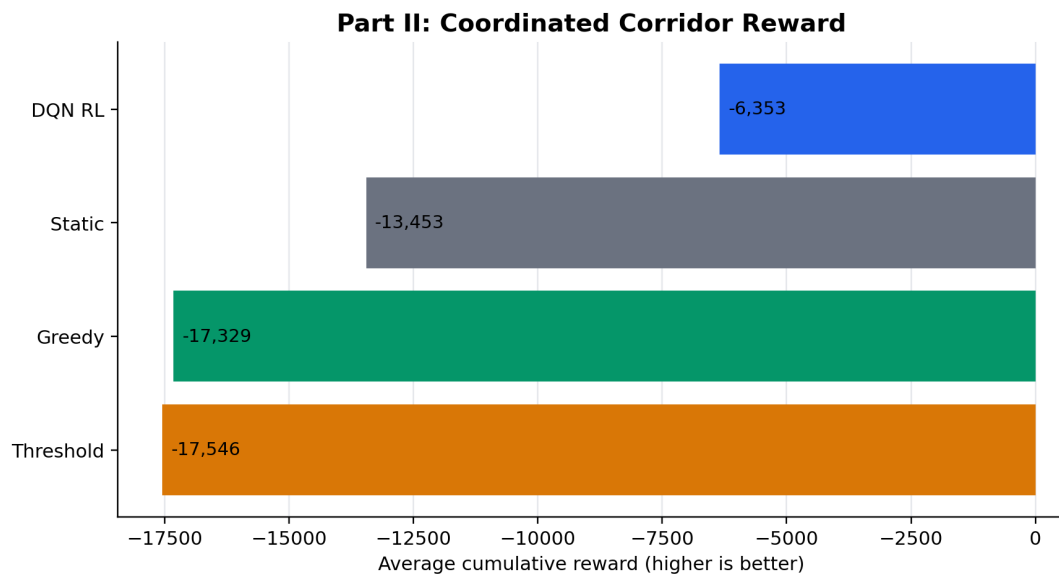


Figure A5. Part II Delay Comparison

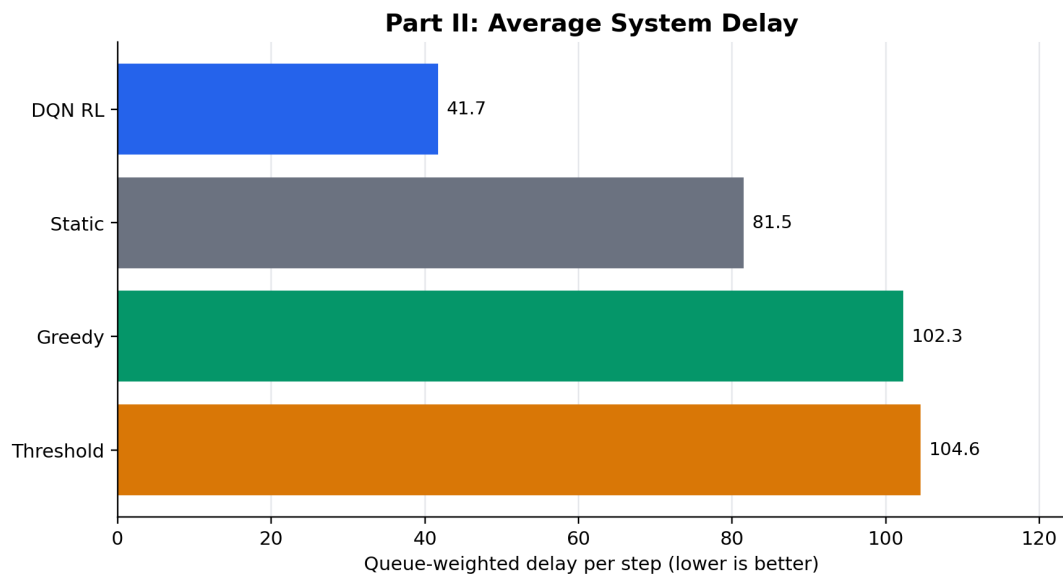


Figure A6. Part II Queue Comparison

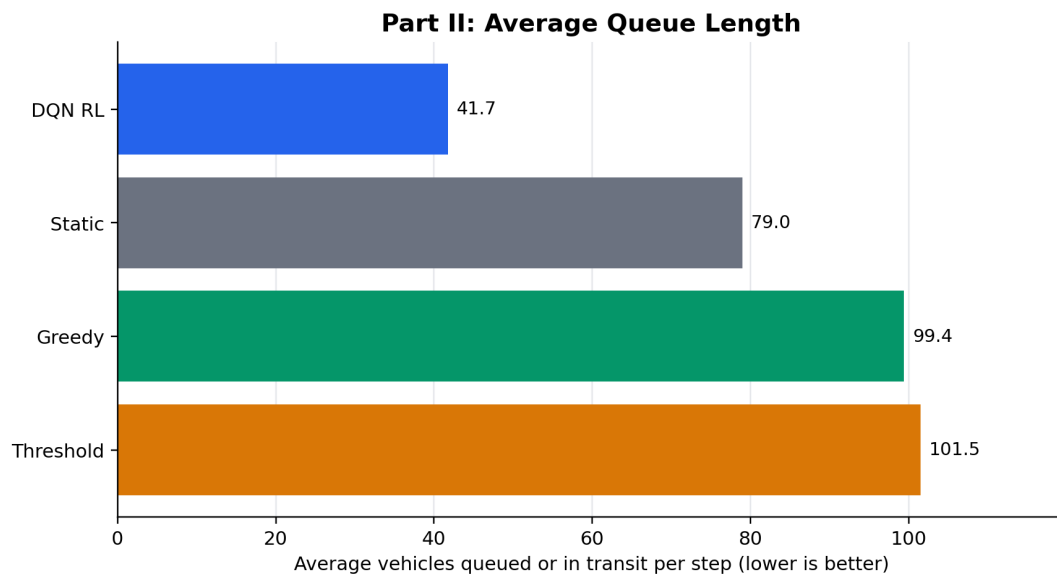


Figure A7. Part II Throughput and Switching Tradeoff